

90 open items, closed.

The whole remaining queue is done: the 1,730-line Home screen is split, every review item is resolved, and the score has been recalculated. Ten real bugs turned up along the way — most of them things a user would have noticed. One change needs checking on a real phone before you ship it.

OPEN ITEMS	APP TESTS	MECHANICAL SCORE	STRICT SCORE
90 → 0	282 → 499	79.7 → 96.1	72.1 → 78.6
44 review findings, 46 mechanical	plus backend, 55 → 70	freshly measured	held down by stale ratings — see below

BUGS FOUND WHILE CLEANING

These weren't on the list. They surfaced because splitting a file forces you to read it properly. Each one is described by what you'd have seen, not by what the code did.

USERS SAW IT

Every comment showed the wrong time

Comment timestamps were saved in UTC but read back as if they were local time, so every comment was off by your timezone offset — three hours out in East Africa, eight in California. Timestamps are now stored the same way as bookmark and download dates.

USERS SAW IT

The AI comparison feature was switched off in every correct build

Bookmarks refused to generate a comparison unless an OpenAI key was present in the app. That key had already been moved to the server for security — so in any properly configured build the variable is empty and the feature simply declined to run. The check is gone; the server owns that decision.

USERS SAW IT

One phone could like the same solution over and over

On the Bookmarks screen, every tap of the thumbs-up added another like, with no record that you'd already liked it. Everywhere else in the app it toggles. All four screens now share one implementation, so the count can't be run up.

USERS SAW IT

Unknown cost and complexity were labelled "Moderate"

Three different places mapped an unrecognised value to the middle of the scale, so a guess was presented as a specific answer — including in the documents users export and keep. The drawer now shows a dash, and exports say "Not specified".

USERS SAW IT

Comparison summaries could stop mid-word

The AI was given a hard length limit but only *asked* to be brief, so a long reply got sliced wherever the budget ran out and the fragment was shown as written. It now trims back to the last finished sentence.

USERS SAW IT

Reduced motion was ignored while the app started

The setting is read from storage, and until it arrived the app assumed motion was fine — during the launch sequence, which is when it runs the most animation at once. Someone who had asked for reduced motion got the animations anyway. It now stays still until it knows otherwise.

COULD HAVE HUNG

A search with no results could spin forever

When a search finds nothing, the app fetches hotlines to offer instead. If the server ever reported "more results available" while returning an empty page, that fetch looped indefinitely, asking for the same page over and over. An empty page now ends the loop regardless of what the flag says.

QUIET DATA LOSS

Liking something while more results loaded undid the like

The "load more" handler held a stale copy of the results list, so appending the next page reverted any like registered in between.

QUIET DATA LOSS

A summary section could be swallowed by the one above it

The comparison summary is split into Use Case, Approach and Complexity/Cost. If one heading came directly after another with no blank line, the second was never recognised and its text was folded into the previous section. Two more spellings the AI commonly uses were also unrecognised and rendered as raw capitals.

SPEED

Opening a category made 152 database calls instead of 7

Loading a list of 30 solutions issued five separate database queries for every single row. It now fetches everything in seven queries no matter how many rows come back.

THE BIG SPLIT

Four files were doing several jobs each. Nothing about how the app behaves changed — the point was that these parts could not be tested or understood separately.

HomeScreen.js 1,730 lines, 45 pieces of state → 263

```
HomeScreen.js      the shell: which half you're on, and what spans both
  home/SearchMode.js  the landing page and the results list
  home/ExploreMode.js stats, category grids, hubs, recent solutions
  home/DrilldownView.js a filtered slice of the catalogue
+ five hooks holding search, explore data, drilldown, help lookup
  and the shared bookmark/like/download behaviour
```

db.js 1,040 lines → 355, as five modules

```
connection.js  opening the database, and the tables this app adds
db.js          the read-only catalogue queries
enrich.js      turning a raw row into a solution
engagement.js  likes, comments, cached summaries – the only writes
heatmaps.js    the two grids
```

BookmarksScreen.js 585 → 215 · **FilterPanel.js** 709 → 492

The 350-line comparison feature moved out of Bookmarks, where it had nothing to do with listing bookmarks. In the filter panel, three separate patterns had each been written out two to four times — the category pickers, the readiness and adoption sliders, and the toggle-chip grids. Several of those copies had already drifted apart from each other.

THE TESTS THAT WERE IMPOSSIBLE BEFORE

The previous session wrote six tests that drive a search from typing to results, watched them fail, and deleted them — the submitting button only appeared after an interaction locked inside the screen's own state, so no test could reach it. That was the argument for splitting the file.

Those tests are back and passing. A search now runs end to end in the suite: typed query, backend call, ordered results, error, retry, and the empty-result hotline state. The suite went from 282 to 499, and the new tests are mostly of things that were previously only assumed — including one that pins the filter contract across the three separate modules that each interpret it, which nothing had checked before.

WHAT THE SCORE ACTUALLY SAYS

The queue is empty, so the tool recalculated. Two of the four numbers are worth separating, because only some of them were re-measured.

SCORE	BEFORE	NOW	BASIS
Mechanical	79.7	96.1	measured today
Overall	75.3	79.4	mixture
Strict	72.1	78.6	75% stale ratings

Only the mechanical score is measured. The tool counts file sizes, duplication and lint directly, so 96.1 is a fact about the code as it stands today — it moved because the things it counts actually changed.

The strict score is three quarters made up of sixteen *judgement* ratings — elegance, naming, test strategy and so on. Nothing can compute those, so the tool has a language model grade them. Every one is still flagged `stale` : they were graded before this work started and have not been re-graded, so the percentages describe code that no longer exists, even though every issue behind them is resolved.

So 78.6 understates where things stand, and the honest reading is "96.1 on what can be measured, unknown on what can't." Re-grading means `desloppify review` , which runs a fresh model-driven review — that would likely raise those sixteen ratings, and would also raise a new batch of findings right after the queue reached zero. That's a deliberate not-yet rather than an oversight.

NEEDS CHECKING ON A REAL DEVICE

Everything below passes in the test suite, but the suite cannot prove these particular things. They're worth ten minutes on a phone before shipping.

```
git fetch && git checkout krishnav8
npm install          # two packages were removed, so this matters
npm start
# the AI search, comparison and dictation features also need:
cd backend && npm install && npm run start
```

First launch on a clean install — the one that matters

The app copies its bundled database into storage the first time it runs. That code moved to the current Expo file API, and **no test can verify it**: every test replaces the file system with a stand-in, so passing tests only prove the new code agrees with a stand-in I also wrote. If it's wrong, the app can't open its database at all — it fails at startup, not subtly.

Delete the app, install fresh, launch. If the catalogue loads, it works. The change is isolated in its own commit, so it's a clean revert if not.

02 Export a solution to a file

The other half of the same file-system change. Tap download on any solution and confirm the share sheet opens with a readable file.

03 Navigation in a release build

The app was disabling native navigation screens in every build to work around a problem that only affects Expo Go. That's now limited to Expo Go, so a release build gets native screens for the first time. Worth tapping through the tabs and back gestures once.

04 Text size set to Large

About 150 pieces of text that previously ignored this setting now follow it. That's the intended fix, but it means some screens will lay out differently at the large size than they ever have.

DELIBERATELY LEFT ALONE

- **The OpenAI key is out of the app, but not yet rotated.** Nothing in the app reads it any more, and the unused `EXPO_PUBLIC_OPENAI_API_KEY` entry has been deleted — the same key already lives in `backend/.env`, where it belongs, so nothing was lost. What remains is that any build made while it was in the app still contains it, on whatever devices have those builds. Only rotating the key at OpenAI fixes that, and your standing instruction was not to, so I haven't.
- **The backend is not locked down yet.** It now supports requiring a client token, but that stays off until you set `API_CLIENT_TOKEN` on the server — turning it on without that would break builds already in users' hands.
- `assets/db/atiokb.db` was never touched, and I checked after every commit.

- **Pushed to krishnav8** — 36 commits, of which 16 are this round of work and the rest were waiting from the previous one. Each is self-contained with the reasoning in the message, so anything here can be reverted on its own. Nothing was merged to **main**.

499 app tests · 70 backend tests · 0 lint problems branch krishnav8 · pushed
database asset unchanged